

Déploiement : une API, ses tests, et la surveillance des données

Le modèle sort du notebook. Il est désormais interrogeable par une API sécurisée et conteneurisée avec son interface, couverte par des tests, et surveillée : un indicateur mesure chaque jour si le marché ressemble encore à celui sur lequel le modèle a appris.

Dépôt DataScientest-Studio/jul26_bde_crypto Date 18 septembre 2026, mis à jour le 23 septembre

11	149	4	15
ROUTES EXPOSÉES PAR L'API	TESTS AUTOMATISÉS	CONTENEURS POUR LE PRODUIT	JEUX SURVEILLÉS EN DÉRIVE

01 En bref

CE QUE FAIT LE SYSTÈME AUJOURD'HUI

On lui demande une paire et un style de trading ; il répond **acheter, vendre** ou **attendre**. Tout tourne dans Docker : les deux bases, l'API, et une interface web qui affiche les bougies en direct avec les décisions du modèle. L'API n'est joignable qu'avec une clé, que seule l'interface détient, côté serveur. Un indicateur de dérive compare les données du marché à celles de l'entraînement.

Ce qui était demandé, et où le trouver

DEMANDE	RÉALISÉ	OÙ
API du modèle et des bases	OUI 11 routes protégées par clé, documentation interactive	api/
Tests unitaires de l'API	OUI 149 tests, plus 21 vérifications du déploiement	tests/, scripts/verifier_deploiement.py
Conteneurisation	OUI API, interface et bases, une seule commande	Dockerfile.api, interface/, docker-compose.yml
Mesure de la dérive	OUI indice PSI, en API et en ligne de commande	api/derive.py, scripts/mesurer_derive.py

```
python -m scripts.generer_secrets # une fois : clés et mots de passe aléatoires dans .env
docker compose up -d --build      # tout démarre
```

L'interface répond alors sur localhost:8080, et la documentation interactive de l'API sur localhost:8080/api/docs, où chaque route s'essaie depuis le navigateur.

02 Le machine learning, en simple

Cette section résume, sans vocabulaire technique, ce que le modèle fait et ce qu'il vaut. Le détail complet est dans le rapport de l'étape 3 et dans le notebook.

La question posée au modèle

Toutes les 15 minutes, 1 heure ou 4 heures, le marché produit une « bougie » : un résumé de ce qui s'est passé pendant cette période (prix d'ouverture, plus haut, plus bas, clôture, volume échangé). La question est : **la prochaine bougie sera-t-elle en hausse ou en baisse ?**

Pour répondre, le modèle reçoit 35 indicateurs calculés sur les bougies précédentes : écart aux moyennes mobiles, RSI, MACD, volatilité, volumes, pression acheteuse, plus l'état des échelles de temps plus lentes. Il ne voit jamais le prix en euros, seulement des rapports — c'est ce qui lui permet de travailler indifféremment sur le Bitcoin à 80 000 \$ ou le XRP à 3 \$.

Le bouton conservateur / agressif

Le modèle ne répond pas « oui » ou « non » mais donne une **probabilité** de hausse. Le style choisit à partir de quelle probabilité le bot agit réellement. En dessous, il répond *attendre*. C'est tout le bouton : un seuil.

STYLE	ACHÈTE SI LA PROBABILITÉ DÉPASSE	VEND SI ELLE DESCEND SOUS	ORDRES PAR JOUR	BONNES RÉPONSES	MARGE D'ERREUR
Agressif	0,5346	0,4315	25	0,597	0,557 – 0,636
Conservateur	0,5415	0,3921	20	0,599	0,554 – 0,642

POURQUOI DEUX SEUILS ET NON UN SEUL

Les probabilités du modèle ne sont pas symétriques : elles descendent bas du côté baisse, mais **ne dépassent jamais 0,564** du côté hausse. Avec un seuil unique mesuré depuis 0,5, le style conservateur se retrouvait au-dessus de ce maximum : il ne pouvait *structurellement* jamais acheter, et ne produisait que des ventes. Le défaut s'est vu tout de suite sur l'interface. Chaque côté a donc désormais son propre seuil, réglé pour déclencher autant de signaux. Le modèle n'était pas baissier — les hausses réelles sont à 50 % — il était simplement plus sûr de lui en annonçant une baisse.

Ces chiffres sont mesurés sur **14 831 bougies postérieures aux données d'entraînement**, collectées après coup et jamais vues par le modèle. C'est la seule mesure qui compte : elle montre qu'il n'a pas appris le passé par cœur.

Le résultat, sans l'enjoliver

LE MODÈLE A RAISON PLUS SOUVENT QU'IL N'A TORT, ET IL PERD QUAND MÊME DE L'ARGENT

Sur une bougie de 15 minutes, le prix bouge en moyenne de **0,227 %**, alors qu'un aller-retour coûte **0,2 %** de frais. Même un modèle qui ne se tromperait jamais ne gagnerait que 0,027 % par ordre. Il faudrait 94 % de bonnes réponses pour seulement couvrir les frais ; nous en sommes à 60 %.

Le backtest le confirme : sur la période récente, le style agressif perd 5,0 % et le conservateur 3,6 %, frais compris. Nous avons cherché à corriger ce point pendant deux semaines — horizons plus longs, carnet d'ordres des contrats à terme, valeurs des bougies précédentes, calibration des probabilités, prédiction de l'amplitude du mouvement. Une seule piste a apporté un gain régulier (la calibration), et aucune n'a rendu le bot rentable.

Ce n'est pas un échec de la chaîne technique, mais un résultat mesuré, conforme à la littérature : la volatilité d'un marché se prédit, sa direction non. C'est pourquoi l'API affiche un avertissement dans chacune de ses réponses.

03 L'API

ROUTE	CE QU'ELLE FAIT
GET /health	L'API, le modèle et les deux bases répondent-ils ? Sert aussi de sonde à Docker
GET /modele	D'où vient le modèle, sur quelles données, ses deux seuils, ses performances
GET /paires	Les paires disponibles en base
GET /bougies/{paire}	Les dernières bougies, lues dans PostgreSQL
GET /couverture	Volume et retard de collecte de chaque jeu de données
POST /prediction	Acheter, vendre ou attendre, selon la paire et le style
GET /derive	Les données récentes ressemblent-elles à celles de l'entraînement ?
GET /graphique/{paire}	Les dernières bougies avec la décision du modèle pour chacune
GET /ordre/{paire}	L'ordre projeté : prix d'entrée, take profit, stop loss
POST /positions/{paire}	Un tour du carnet de positions virtuelles : ferme, ouvre, fait le bilan
GET /metrics	Métriques pour la supervision, invisibles depuis l'extérieur

Un appel, une réponse

```
curl -X POST http://localhost:8080/api/prediction \
-H "Content-Type: application/json" \
-d '{"symbole": "BTCUSD", "interval": "1h", "style": "conservateur"}'
```

```
{ "symbole": "BTCUSD", "interval": "1h", "style": "conservateur",
  "bougie": "2026-09-23T09:00:00+00:00",
  "probabilite_hausse": 0.5346, "seuil_achat": 0.5415, "seuil_vente": 0.3921,
  "decision": "attendre", "sens": 0,
  "avertissement": "modele experimental, non rentable frais compris" }
```

Réponse réelle, obtenue à travers le proxy le 23 septembre. Le modèle penche vers la hausse (0,5346) mais reste sous son seuil d'achat (0,5415) : il s'abstient. C'est le comportement attendu la plupart du temps — en style conservateur, le bot ne se prononce que 20 fois par jour sur les cinq paires.

Une interface pour voir le modèle travailler

`http://localhost:8080` affiche les bougies en direct, le prix qui bouge, la décision du modèle et le carnet de positions virtuelles : une flèche à chaque ouverture, un point vert ou rouge à chaque fermeture, les zones de take profit et de stop loss en rectangles. Le bouton conservateur / agressif se change dans la page, et l'historique remonte jusqu'à un an.

Deux choix rendent cette page simple et sûre :

- **Elle a son propre conteneur**, un serveur nginx qui sert la page et transmet ses appels à l'API sous `/api`. Page et données ont la même adresse : aucun réglage d'origine (CORS) à ouvrir. Et c'est nginx qui ajoute la clé d'API, jamais le navigateur (voir plus bas).
- **Elle lit Binance en direct**. La route `/graphique` va chercher les bougies fraîches et fait prédire le modèle dessus. La bougie en cours est affichée mais jamais prédite : sa clôture n'existe pas encore.

Le graphique utilise Lightweight Charts, la bibliothèque libre de TradingView, **livrée avec le projet** plutôt que chargée depuis Internet : la page fonctionne sans dépendance extérieure, et aucun site tiers ne peut y injecter de code.

Trois choix à défendre

- **L'API lit les vues, jamais les tables**. `v_day_trading` restitue les trois pas de temps du profil, où qu'ils soient physiquement stockés. C'est exactement la raison d'être de ces vues, décidée à l'étape 2.
- **Le modèle est chargé une fois et gardé en mémoire**. Le fichier pèse 99 Mo ; le relire à chaque appel rendrait l'API inutilisable. Quand un nouveau modèle remplace le fichier, l'API le voit à sa date de modification et le recharge seule, sans redémarrer.
- **Chaque prédiction est archivée** dans la table `api_predictions`. Sans cette trace, impossible de répondre plus tard à « le modèle s'est-il mis à vendre beaucoup plus qu'avant ? », qui est le premier signe visible d'une dérive.

La sécurité

Une API de modèle expose deux choses à protéger : les données (et notre quota d'appels à Binance, que chaque requête consomme) et l'écriture dans le carnet de positions. La protection tient en quatre couches.

COUCHE	CE QUI EST FAIT	POURQUOI
Qui peut appeler	Une clé d'API par client (interface, Airflow), longue et aléatoire, comparée en temps constant	Chaque appel est attribué à un client ; une clé compromise se révoque sans couper l'autre. La comparaison en temps constant empêche de deviner une clé caractère par caractère en chronométrant les refus.
Par défaut	API fermée : sans clé configurée, elle répond 503	Un oubli de configuration ne doit jamais ouvrir l'API. L'accès libre existe pour le développement, mais doit être demandé explicitement.
Où est la clé	Ajoutée par le proxy nginx de l'interface, jamais envoyée au navigateur	Tout ce qui arrive dans le navigateur peut être lu par n'importe qui : une clé écrite dans la page ne protégerait rien.
Qui peut joindre l'API	Aucun port publié : seuls nginx, Airflow et Prometheus la joignent, dans le réseau Docker	L'interface, seule porte d'entrée, limite le débit (10 requêtes par seconde et par adresse) et masque <code>/metrics</code> .
Ce qui entre	Paires, pas de temps et styles validés par liste blanche	Une paire inconnue est refusée avant tout appel à Binance.

S'y ajoutent des en-têtes de sécurité (politique de contenu, interdiction d'afficher la page dans un cadre), des conteneurs qui tournent **sans droits root**, et des secrets générés aléatoirement par `scripts/generer_secrets.py`, jamais versionnés.

POURQUOI DES CLÉS, ET PAS DES JETONS JWT

Un jeton JWT répond à un autre problème : des *personnes* qui se connectent avec un mot de passe et reçoivent un laissez-passer temporaire. Les clients de notre API sont des *services*, sans utilisateur derrière. Une clé longue, tenue hors du navigateur et propre à chaque client, est la réponse standard à ce cas, celle des API de Binance, de Stripe ou de GitHub.

04 Les tests

149 tests, exécutés en une dizaine de secondes par `pytest tests/ -q`, et relancés automatiquement à chaque envoi de code sur GitHub (étape 5).

FAMILLE	NOMBRE	CE QU'ELLE VÉRIFIE
API	59	Chaque route, les décisions des deux styles, les codes d'erreur (401, 404, 422, 503), le carnet de positions
dont sécurité	16	Refus sans clé ou avec une fausse clé, API fermée si rien n'est configuré, une clé par client, liste blanche. Un test parcourt <i>toutes</i> les routes : une route ajoutée sans protection le fait échouer.
Pipeline et supervision	23	Contrôle qualité, ingestion, règle de promotion d'un modèle, rechargement à chaud, métriques (étape 5)
Variables explicatives	4	Aucun regard vers le futur, indépendance de la fenêtre de calcul et du niveau de prix
Étiquetage	19	Barrières, absence de fuite temporelle (étape 3)
Données et configuration	44	Nettoyage, cohérence entre le code et le schéma SQL (étapes 1 et 2)

Le principe : aucun test ne dépend des bases ni du vrai modèle

Les accès extérieurs sont remplacés par des doublures — des fausses bougies et un faux modèle qui renvoie une probabilité choisie. Trois raisons :

- les tests passent sur la machine d'un collègue, **sans Docker** ;
- ils durent quelques secondes au lieu de plusieurs minutes (le vrai modèle pèse 99 Mo) ;
- un test qui échoue désigne un bug de l'API, **pas une base éteinte**.

Exemple de test parlant : en réglant le faux modèle sur une probabilité de 0,58, on vérifie que le style agressif achète et que le conservateur s'abstient. C'est le cœur du bouton, vérifié automatiquement à chaque modification.

Le code central est bien couvert : 98 % des lignes de la sécurité, 91 % du chargement du modèle, 82 % des routes. Les modules qui ne font que lire et écrire en base le sont moins par les tests unitaires ; ils sont vérifiés autrement, sur l'application démarrée.

Ce qu'un test unitaire ne peut pas voir

Que la clé soit bien ajoutée par nginx, que l'API soit vraiment injoignable de l'extérieur, que la limitation de débit fonctionne : cela n'existe qu'une fois les conteneurs assemblés. `scripts/verifier_deploiement.py` le vérifie **de l'extérieur, comme un utilisateur** — 21 vérifications, dont une rafale de 120 appels dont environ 75 sont refusés par nginx.

05 La conteneurisation

SERVICE	RÔLE	ACCÈS DEPUIS LA MACHINE
postgres	TimescaleDB : bougies nettoyées, référentiel, journaux	<code>127.0.0.1:5432</code>
mongo	Couche brute : réponses de l'API Binance intactes	<code>127.0.0.1:27017</code>
api	Le modèle et les données, exposés en HTTP	aucun (réseau Docker seulement)
interface	La page et le proxy vers l'API (nginx)	<code>localhost:8080</code>
pgadmin, mongo-express	Regarder les bases, pour une démonstration	<code>127.0.0.1:5050</code> , <code>:8081</code>

Tous les ports sont liés à `127.0.0.1` : joignables depuis cette machine, jamais depuis le réseau. L'étape 5 ajoute Airflow, MLflow et la supervision dans le même fichier, rangés en **profils** qu'on active ou non.

Quatre décisions utiles

- Une image dédiée, plus légère.** L'API n'a besoin ni de MLflow, ni de Jupyter, ni de matplotlib : ses dépendances sont listées à part dans `requirements-api.txt`.
- Le modèle n'est pas copié dans l'image**, il est monté depuis `./models` en lecture seule. Un nouveau modèle se met en service sans reconstruire l'image, ni même redémarrer l'API.

- **L'interface a son conteneur à elle**, un nginx de 82 Mo qui tourne sans droits root. Elle ne partage que le réseau « services » avec l'API : même compromise, elle ne peut pas atteindre une base de données, qui vivent sur un autre réseau.
- **Chaque service attend que ceux dont il dépend soient sains** avant de démarrer, et expose sa propre sonde de santé : Docker redémarre ce qui cesse de répondre.

Un piège rencontré : **l'image contient le code de calcul des indicateurs**.

Après avoir corrigé un indicateur, nous avons d'abord oublié de la reconstruire, et l'API a servi pendant quelques minutes des variables différentes de celles du modèle. La commande à retenir est `docker compose up -d --build api` après toute modification de `src/`.

06 La dérive des données

LE PROBLÈME, EN UNE PHRASE

Le modèle a appris sur une certaine période. Si le marché change — volatilité qui double, volumes qui s'effondrent, nouveau régime — les données qu'il reçoit ne ressemblent plus à celles de son apprentissage, et ses réponses deviennent douteuses. **Rien ne plante : c'est précisément le danger.**

Comment on la mesure

Pour chaque variable, on a photographié la répartition de ses valeurs pendant l'entraînement, découpée en 10 tranches de taille égale. On regarde ensuite où tombent les valeurs récentes dans ces mêmes tranches. Si la répartition est identique, chaque tranche reçoit 10 % des bougies ; si tout s'entasse d'un côté, la variable a dérivé. L'écart se résume par un indice appelé **PSI**, d'usage courant dans l'industrie :

INDICE PSI	LECTURE	ACTION
moins de 0,10	stable	rien à faire
0,10 à 0,25	dérive modérée	surveiller
plus de 0,25	dérive forte	réentraînement conseillé

Deux règles, apprises en construisant la mesure

- **On ne compare que des choses comparables.** La référence est choisie pour la paire et le pas de temps. La taille moyenne d'un trade vaut 0,005 sur le Bitcoin et 115,8 sur le XRP ; l'amplitude d'une bougie 4h vaut quatre fois celle d'une bougie 15 minutes. Comparer à une référence mélangée annonçait une dérive massive alors que rien n'avait bougé.
- **La fenêtre se donne en jours, pas en nombre de bougies.** Trop courte, elle fait sortir en « dérive forte » les variables lentes, simplement parce que quelques jours ne couvrent qu'une petite partie de leur distribution habituelle. Trois mois est le réglage retenu.

Mesure du 23 septembre, sur 90 jours

PAIRE	15 MIN	1 H	4 H	VARIABLES LES PLUS CONCERNÉES
BTCUSDT	3 FORTES	5 FORTES	7 FORTES	volatilité (ATR) du contexte 4h
ETHUSDT	9 FORTES	9 FORTES	12 FORTES	volatilité (ATR), à toutes les échelles
BNBUSDT	7 FORTES	7 FORTES	8 FORTES	volatilité, taille moyenne d'un trade
SOLUSDT	11 FORTES	10 FORTES	14 FORTES	taille moyenne d'un trade, volatilité
XRPUSDT	7 FORTES	6 FORTES	8 FORTES	taille moyenne d'un trade, volatilité

CORRECTION PAR RAPPORT À LA PREMIÈRE VERSION DE CE RAPPORT

La mesure du 18 septembre donnait le Bitcoin stable et les pas de temps longs peu touchés. Elle était faussée à deux titres. D'abord, le script en ligne de commande lisait assez de bougies pour couvrir 90 jours en 15 minutes, mais **ne coupait pas la fenêtre des autres pas de temps** : le 1h était mesuré sur 360 jours et le 4h sur deux ans, ce qui noie tout changement récent. Ensuite, la base s'arrêtait au 29 août. Les deux défauts sont corrigés : la fenêtre est la même durée pour chaque pas de temps, et la base est alimentée en continu (étape 5).

Toutes les séries dérivent fortement, et la mesure désigne surtout la *volatilité* : le marché des trois derniers mois ne bouge pas avec la même amplitude que pendant l'apprentissage du modèle (fin août 2024 à début février 2026). La *taille moyenne d'un trade*, seule variable qui dépende de l'unité de la paire, reste la deuxième coupable, et la première candidate à une révision. C'est exactement la situation que le réentraînement automatique de l'étape 5 doit traiter.

La mesure s'exécute aussi en ligne de commande, et chaque exécution est archivée dans `docs/derive_donnees.json` : ce qui compte n'est pas la valeur d'un jour, mais sa **tendance**. C'est cette commande qu'Airflow appelle désormais chaque matin (étape 5).

07 Limites et suite

LIMITE CONNUE	CONSÉQUENCE	PISTE
Le modèle n'est pas rentable	L'API est un démonstrateur, pas un outil d'investissement	Cible ignorant les mouvements trop petits pour couvrir les frais
Fichier de modèle de 99 Mo	Démarrage de l'API en quelques secondes	Réduire le nombre d'arbres, à performance quasi identique
Une variable dépend de l'unité de la paire	Première source de dérive détectée	La rapporter à sa propre moyenne récente
Le marché a changé de régime de volatilité	Toutes les séries dérivent fortement	Réentraînement automatique, avec un duel contre le modèle en service (étape 5)

Ce que l'étape 5 a apporté

Collecte continue des bougies toutes les 15 minutes, bot qui tourne sans que personne ne regarde, mesure de dérive chaque matin, réentraînement chaque semaine avec un duel contre le modèle en service, intégration continue et supervision. Le détail est dans le rapport de l'étape 5.

Code `api/`, `interface/`, `tests/`, `Dockerfile.api` Mesures `docs/derive_donnees.json`, `docs/modele_final_direction.json`
Suivi MLflow, expérience `cryptobot_direction`